

51515

Inventor

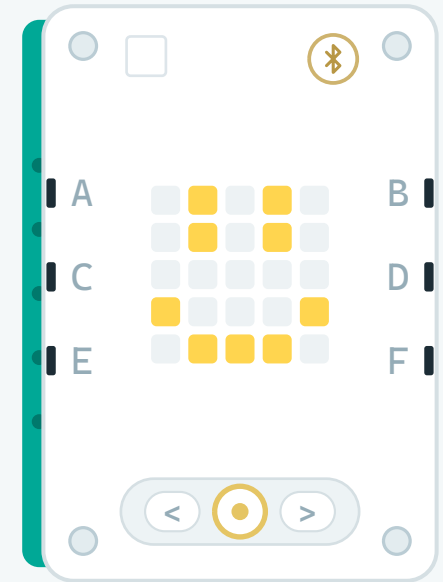
Pybricks

Lego MINSTORMS Inventor

Basics: Setup, Patterns, and Resources.

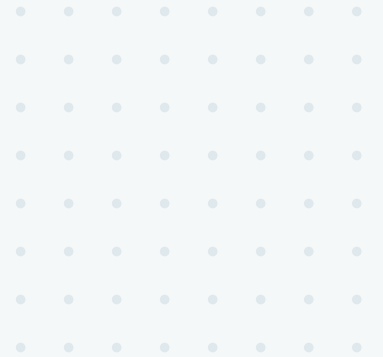
Robotika RKA | Tutorial #1

21 September 2026



01

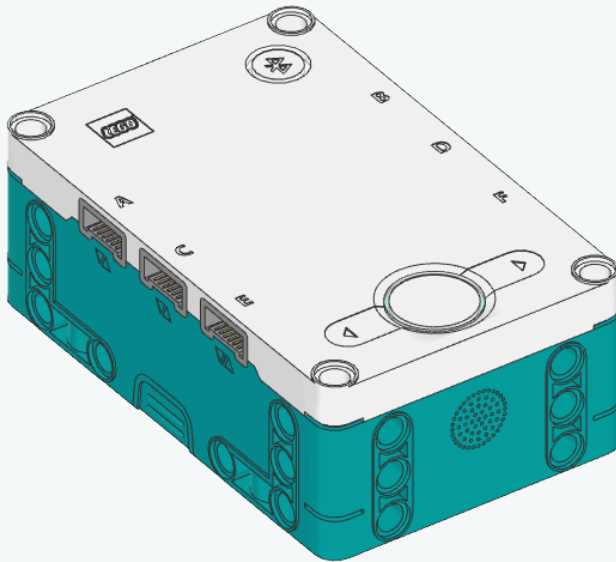
Hardware Setup



Inventor Hub

Inventor Hub adalah komponen utama yang memuat orkestrasi dari semua sensor dan aktuator pada sebuah robot LEGO Inventor.

Bentukannya kurang-lebih begini:



Di sini itu yang lubang samping-sampingnya itu UART port, dari A ke F. Untuk power input menggunakan microUSB, dan koneksi ke laptop dengan bluetooth.

Di tengah itu tombol power/execution, terus di pojok kanan-atas itu bluetooth.

What's Pybricks?

Inventor Hub ini itu seperti komputer kecil. Di dalamnya ada firmware, sama seperti laptop punya sistem operasi (*but lower level, "technically"*). Firmware bawaan LEGO hanya mau bekerja dengan aplikasi ekosistem LEGO, dan ada beberapa kekurangan juga dari firmware bawaan ini: kurang bisa di-*tweaking* dan ada latensi-latensi yang diakibatkan *bloat*.

Oleh karena itu, pada Hub yang kita akan gunakan, kita ganti firmware ini dengan firmware open-source PyBricks yang memiliki API lebih konsisten, lebih performan, dan lebih *maintained*.

(serta literally tidak discontinued minggu depan lmao 🤪)



Compatibilities

Intendendnya sih kita pake Pybricks Code (<https://code.pybricks.com>), yang menggunakan web bluetooth. Jadi ada beberapa yang perlu diperhatikan untuk kompatibilitas di:

Browser	Win	macOS	Linux	Chrome OS	iOS / iPad
Chrome	bisa	bisa	bisa	bisa	tidak
Edge	bisa	bisa	bisa	bisa	tidak
Chromium	bisa	bisa	bisa	bisa	tidak
Brave	tidak	tidak	tidak	tidak	tidak
Safari	N/A	tidak	N/A	N/A	tidak
Firefox	tidak	tidak	tidak	tidak	tidak

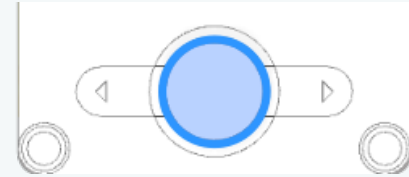
Catatan Penting

- ◆ **Brave** memblokir Web Bluetooth by default via Shields (tidak bisa).
- ◆ **Firefox** tidak mendukung Bluetooth di platform mana pun.
- ◆ **iPad & iPhone** diblokir Apple untuk semua browser.
- ◆ Gunakan **Chromium-based** (kecuali brave 🦋) kalau ingin menggunakan Pybricks Code.

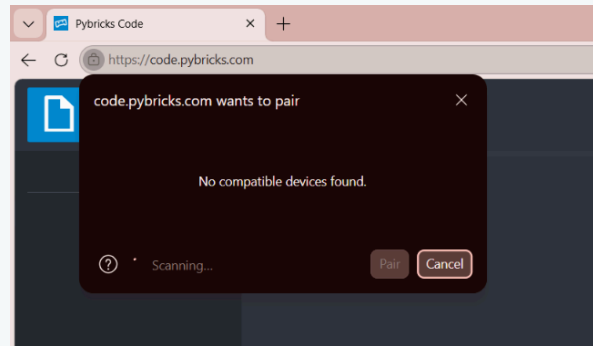
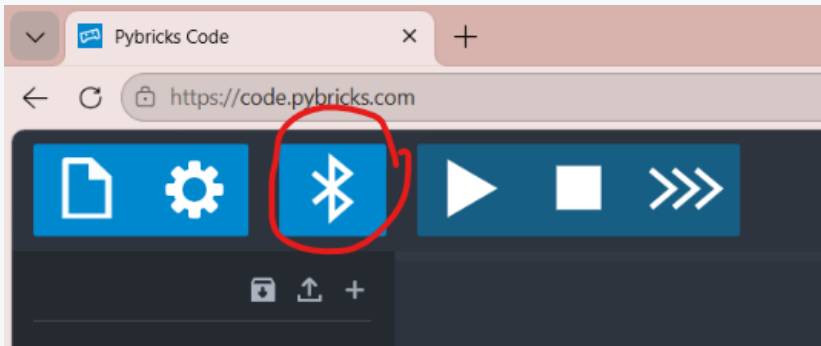
Sebagai alternatif lain, ada juga pakai **BlockLyPy** kalau mau dari VSCode langsung.

Setup Hub

Nyalakan hub dengan menekan tombol tengah, nanti lampunya di sekeliling tombol juga akan nyala. Di tombol bluetooth juga bakal kedip-kedip.



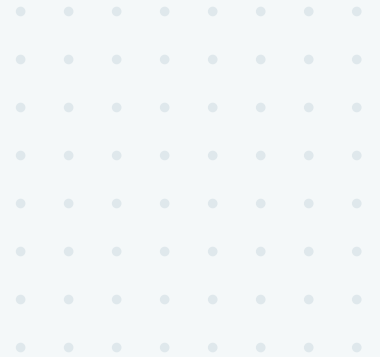
dari sini, pencet tombol ini di PyBricks code, terus nanti pilih device-nya untuk connect.



Kalau berhasil connect, nanti tombol bluetooth akan menyala terus. Kalo salah koneksi dengan device lain, pencet tombol power lama untuk restart lalu reconnect.

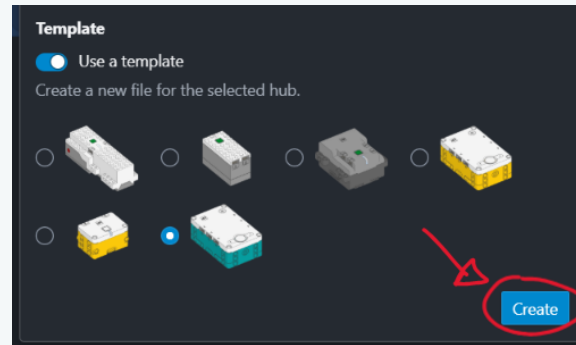
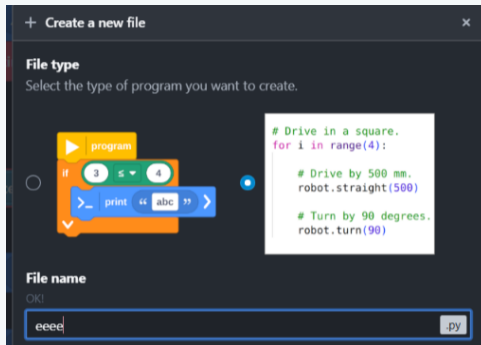
02

Hello World

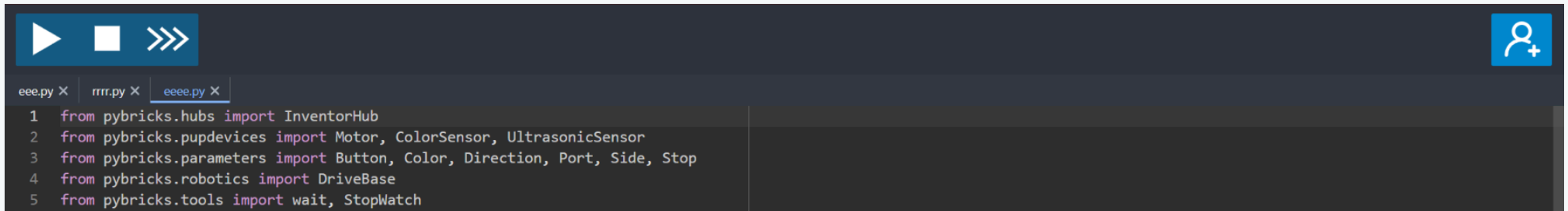


First program

Buka tab file, klik ikon +, pilih Python, beri nama program, lalu klik Create.



Nanti muncul seperti ini:



First program

Isi dengan ini:

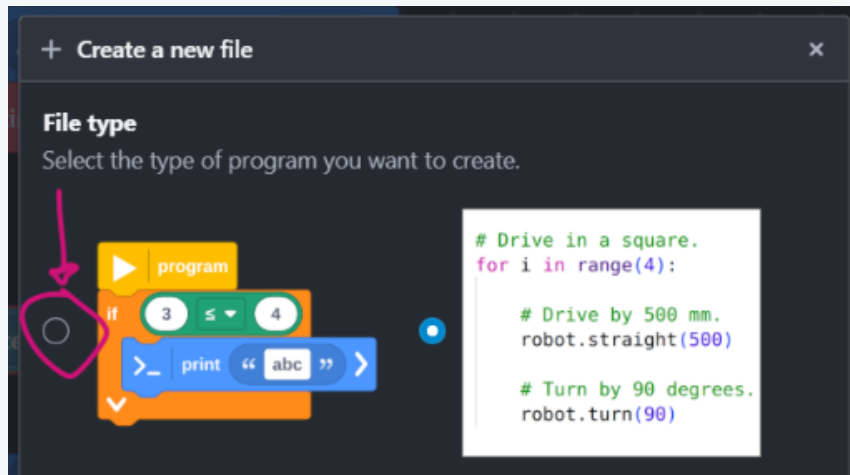
```
1 from pybricks.hubs import InventorHub
2 from pybricks.tools import wait
3
4 hub = InventorHub()
5
6 print("hub siap")
7 hub.speaker.beep()
8 wait(1000)
9 print("baterai:", hub.battery.voltage(), "mV")
```

First program

Klik tombol play. Hub akan berbunyi, dan teks muncul di panel output di bawah editor.

Kalau kedua hal itu terjadi, instalasi berhasil. 🎉

Selain yang python, Pybricks Code juga ada block-based interface, bisa juga sebagai alternatif, sesuai preferensi saja.

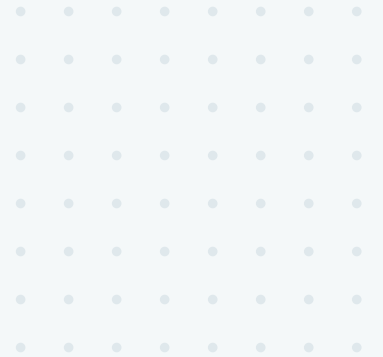


Setelah dijalankan dari aplikasi, programnya akan tersimpan di Hub-nya. Untuk menjalankan kembali, bisa dipencet tombol tengah, nanti akan berjalan lagi.

Hub Inventor punya lima program slot. Tombol yang kiri dan kanan untuk memilih slot, angka slot muncul di layar 5x5. Program yang kalian jalankan dari aplikasi akan tersimpan di slot yang sedang terpilih.

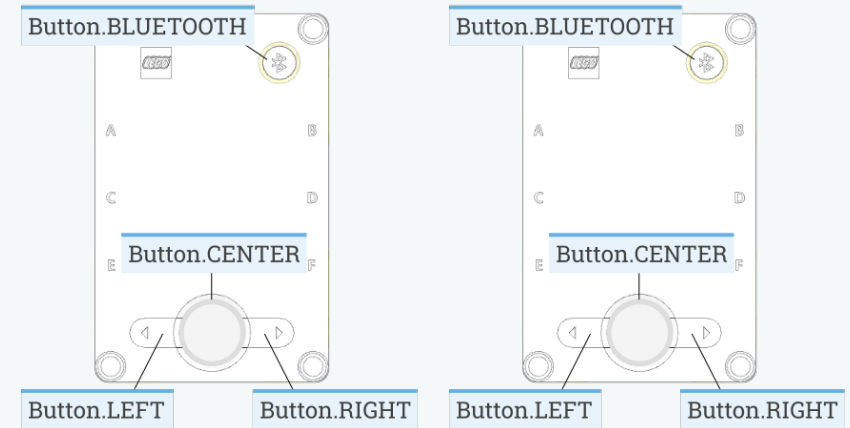
03

Pybricks Basics



Anatomi Hub

Bagian	Jumlah	Keterangan
Port Sensor/Motor	6, dari A sampai F	Bebas dipakai untuk motor maupun sensor.
Layar	25 lampu, susunan 5x5	Untuk menampilkan angka, huruf, dan ikon
Tombol	4	Kiri, kanan, tengah, dan Bluetooth
IMU	1	Mengukur kemiringan dan putaran
Speaker	1	Nada dan bunyi



Motors

Basics

Motor dihubungkan ke port (misal `Port.A`). Perintah gerak menentukan apakah baris berikutnya menunggu (**blocking**) atau langsung lanjut:

Perintah	Satuan	Blocking?
<code>motor.run(speed)</code>	deg/s	Tidak
<code>motor.run_time(...)</code>	deg/s, ms	Ya
<code>motor.run_angle(...)</code>	deg/s, deg	Ya
<code>motor.run_target(...)</code>	deg/s, deg	Ya
<code>motor.dc(duty)</code>	% tegangan	Tidak

```
1 from pybricks.pupdevices import Motor
2 from pybricks.parameters import Port
3
4 motor = Motor(Port.A)
```

Catatan Eksekusi

Perintah dengan satuan waktu/sudut otomatis menahan eksekusi kode sampai motor selesai bergerak.

Motors

Blocking vs Non-Blocking

Pembeda paling krusial adalah bahwa baris berikutnya menunggu putaran selesai atau langsung lanjut saat motor masih jalan.

```
1 motor = Motor(Port.A)
2
3 # 1. Menunggu sampai selesai 360°
4 motor.run_angle(500, 360)
5 print("putaran pertama selesai")
6
7 # 2. Langsung lanjut ke baris bawahnya!
8 motor.run(500)
9 print("ini muncul saat motor jalan")
10 wait(2000)
11 motor.stop()
```

run_angle vs run_target

- ♦ `run_angle(speed, angle)`:
Berputar sejauh sudut dihitung dari **posisi sekarang** (relatif).
- ♦ `run_target(speed, target)`:
Berputar menuju sudut tertentu dihitung dari **titik nol** (absolut).

Keduanya sering tertukar dan efeknya berbeda jauh.

Motors

Stopping

Ada 3 cara berhenti:

Perintah	Yang Terjadi	When to Use
<code>motor.stop()</code>	Listrik dilepas, berputar bebas sampai pelan sendiri	Gerakan bebas, boleh meluncur
<code>motor.brake()</code>	Motor menahan pasif (hubung singkat coil)	Berhenti cepat tanpa presisi
<code>motor.hold()</code>	Motor aktif menahan posisi terakhir	Lengan yang harus tetap terangkat

Lengan Robot Jatuh Sendiri?

Kalau lengan robot lemas setelah program selesai, itu karena berhenti dengan `stop()`, gunakan `hold()`.

`run_angle` dkk. punya argumen `then`:

```
1 from pybricks.parameters import Stop
2
3 # Bawaannya adalah Stop.HOLD
4 motor.run_angle(500, 90,
   then=Stop.HOLD)
```

Encoder (Sensor Sudut)

Setiap motor Powered Up punya sensor sudut (**encoder**) di dalamnya. Motor tidak pernah lupa posisinya selama hub menyala.

```
1 motor = Motor(Port.A)
2 motor.reset_angle(0)
3
4 motor.run(300)
5 wait(3000)
6 motor.stop()
7
8 print("sudut:", motor.angle(), "derajat")
9 print("kecepatan:", motor.speed(), "deg/s")
```

Coba :0

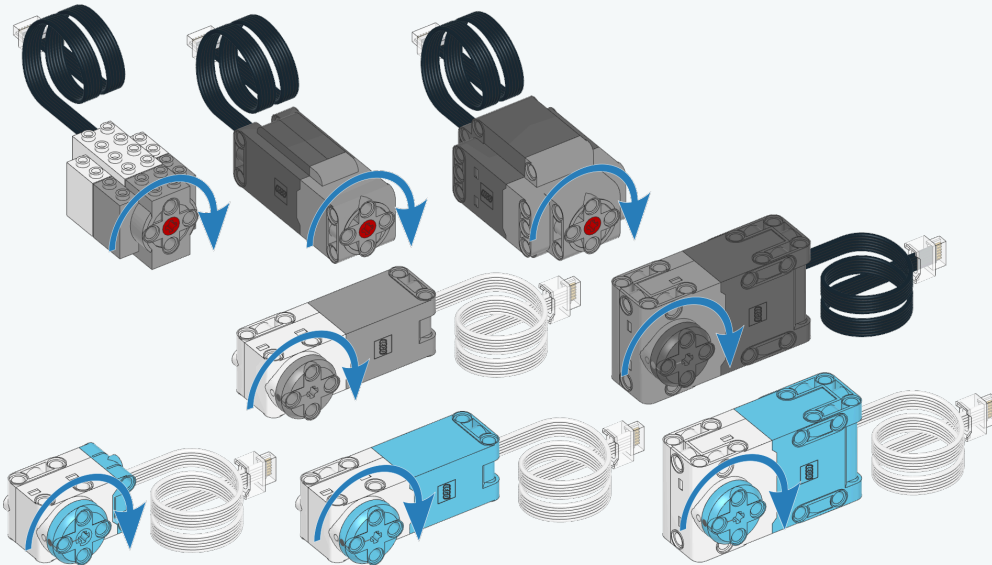
Jalankan kode, lalu coba putar poros motor dengan tangan dan jalankan lagi.

Dasar Odometri

Inilah yang dapat dipakai robot untuk *memperkirakan* sudah berjalan sejauh apa.

Arah Putaran Motor

Setiap jenis motor punya arah positif bawaan (panah biru). Dua motor yang dipasang berhadapan di kiri dan kanan robot akan berputar ke arah berlawanan saat diberi perintah yang sama dan **harus dibalik salah satunya**:



```
1 from pybricks.parameters import Direction
2
3 # Balik arah motor kiri/kanan
4 left = Motor(Port.A,
5               Direction.COUNTERCLOCKWISE)
6 right = Motor(Port.B, Direction.CLOCKWISE)
```

Robot Muter di Tempat?

Kalau robot disuruh maju lurus tapi malah berputar di tempat, artinya arah putaran salah satu motor belum dibalik.

I Deteksi Macet

Motor tahu kalau dirinya ditahan/tertahan.

```
1 motor.run_until_stalled(300, duty_limit=40)
2 motor.reset_angle(0)
```

Ini cara umum mencari titik nol sebuah lengan atau capit. Suruh motor bergerak sampai membentur pembatas mekanis, lalu tetapkan posisi itu sebagai sudut nol. Argumen `duty_limit` membatasi tenaga supaya benturannya tidak merusak gigi.

I Susunan Roda Gigi (Gears)

Untuk motor yang tersambung ke roda lewat gigi, sebutkan susunan giginya supaya perhitungan sudut tetap benar:

```
1 motor = Motor(Port.A, gears=[12, 36])
```

Angka itu jumlah gigi, dari gigi yang menempel di motor sampai gigi yang menempel di roda.

DriveBase

Menggerakkan dua motor secara manual untuk jalan lurus sangat merepotkan. DriveBase mengontrol kedua motor sekaligus dalam satuan **milimeter** dan **derajat**.

```
1 from pybricks.pupdevices import Motor
2 from pybricks.parameters import Port, Direction
3 from pybricks.robotics import DriveBase
4
5 left = Motor(Port.A, Direction.COUNTERCLOCKWISE)
6 right = Motor(Port.B, Direction.CLOCKWISE)
7 robot = DriveBase(left, right, wheel_diameter=56,
8 axle_track=114)
9
10 robot.straight(300) # maju 300 mm
11 robot.turn(90) # putar 90° kanan (CW)
12 robot.straight(-300) # mundur 300 mm
13 robot.turn(-90) # putar 90° kiri (CCW)
```

Konvensi Arah & Sudut

- ◆ **Positif (+):** Maju, atau berputar searah jarum jam (CW / kanan).
- ◆ **Negatif (—):** Mundur, atau berputar berlawanan jarum jam (CCW / kiri).

Parameter

- ◆ **wheel_diameter:** Diameter roda fisik (mm).
- ◆ **axle_track:** Jarak titik tengah kedua roda (mm).

Perintah Lengkap DriveBase

Metode bawaan DriveBase untuk navigasi robot:

Perintah	Satuan	Keterangan
<code>robot.straight(distance)</code>	mm	Maju (+) / mundur (—), program menunggu
<code>robot.turn(angle)</code>	deg	Berputar di tempat (+ kanan, — kiri), menunggu
<code>robot.arc(radius, angle)</code>	mm & deg	Menikung dengan radius tertentu, menunggu
<code>robot.drive(speed, turn_rate)</code>	mm/s & deg/s	Jalan terus tanpa henti, langsung lanjut
<code>robot.stop()</code>	—	Menghentikan pergerakan motor
<code>robot.distance()</code>	mm	Total jarak tempuh menurut encoder roda
<code>robot.angle()</code>	deg	Sudut hadap robot menurut odometri
<code>robot.reset()</code>	—	Kembalikan hitungan jarak & sudut ke nol

How to Calculate

Untuk roda LEGO, diameter sering tercetak di sisi ban. Tulisan 62.4 x 20 berarti diameternya 62.4 mm dan lebarnya 20 mm. Untuk mengukur `axle_track`, jarak antar lubang di balok Technic adalah 8 mm, jadi kalian bisa menghitungnya dari jumlah lubang.

Prosedur Kalibrasi DriveBase

Pengukurannya hampir tidak pernah pas karena ban tertekan bobot robot (diameter efektif mengecil), sehingga axel sedikit melengkung di bawah beban. Kalibrasinya harus dilakukan secara berurutan:

1 Setel wheel_diameter

Jalankan `robot.straight(1000)`, ukur dengan meteran:

- ♦ Kurang dari 1000 mm → **Kecilkan** `wheel_diameter`
- ♦ Lebih dari 1000 mm → **Besarkan** `wheel_diameter`

2 Setel axle_track

Setelah diameter pas, jalankan `robot.turn(360)`:

- ♦ Kurang berputar → **Besarkan** `axle_track`
- ♦ Kelewat berputar → **Kecilkan** `axle_track`

Berurutan

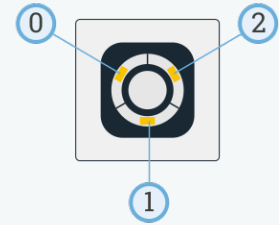
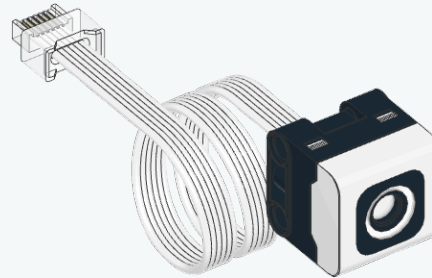
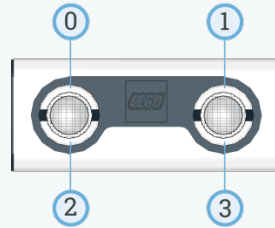
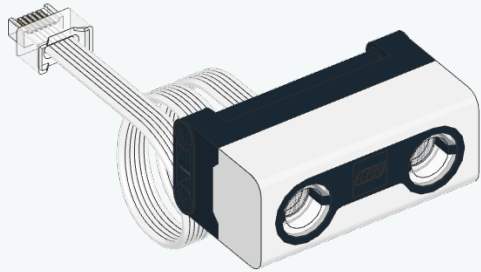
Wajib diameter roda lebih dulu, karena kalkulasi sudut belok robot ikut mengandalkan nilai diameter roda.

Sifat Permukaan Lantai

Kalibrasi di **keramik** akan meleset di **karpét**. Nilai kalibrasi tergantung robot + permukaannya.

Sensors

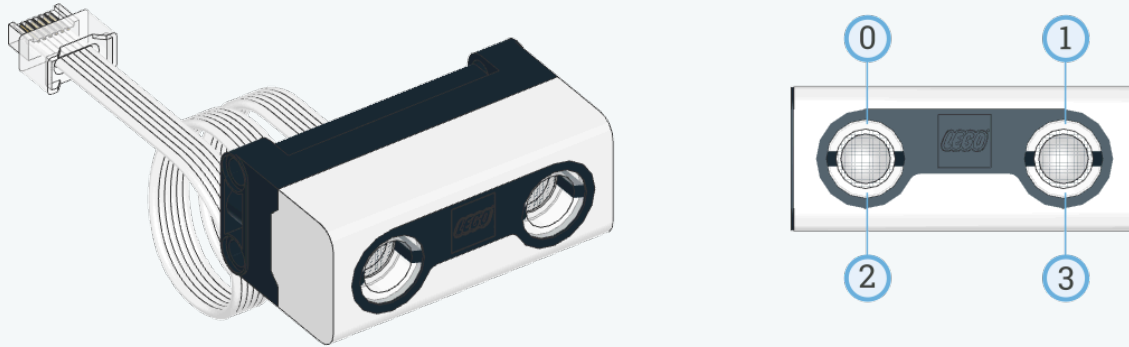
Set 51515 berisi satu Ultrasonic Sensor dan satu Color Sensor.



Kedua sensor ini bisa digunakan untuk mengukur jarak dan menentukan warna yang diobservasi oleh robot.

Ultrasonic Sensor

Fungsi dari ultrasonic sensor adalah mengukur jarak ke benda di depannya dengan gelombang suara.



Di sini, nomor pada gambar kanan adalah urutan lampu. Satu angka menyalakan keempatnya sekaligus, `sensor.lights.on(100)`, dengan empat angka untuk mengatur masing-masing, `sensor.lights.on([100, 100, 0, 0])` (dengan asumsi variabel sensor di-set jadi sensor).

Ultrasonic Sensor

```
1 from pybricks.pupdevices import UltrasonicSensor
2 from pybricks.parameters import Port
3 from pybricks.tools import wait
4
5 eyes = UltrasonicSensor(Port.C)
6
7 while True:
8     print(eyes.distance(), "mm")
9     wait(200)
```

Ultrasonic Sensor

Ultrasonic Methods:

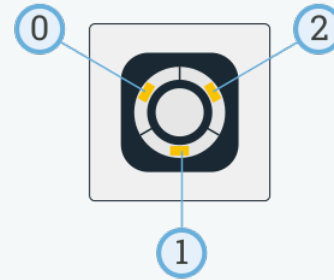
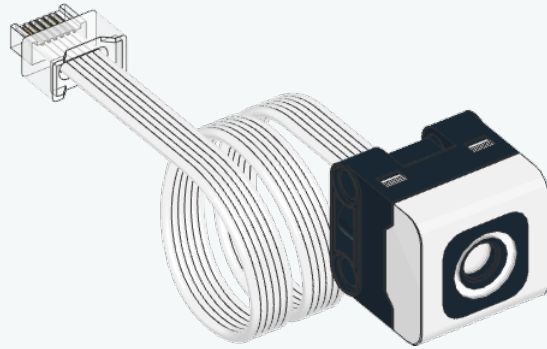
Perintah	Hasil
<code>eyes.distance()</code>	Jarak dalam mm
<code>eyes.presence()</code>	True kalau ada sensor ultrasonic lain di dekatnya
<code>eyes.lights.on(brightness)</code>	Menyalakan empat lampu di sensor

Ultrasonic Notes:

Ketika kalian uji, sensor ini punya beberapa perilaku yang perlu kalian ketahui sebelum mengira kodenya salah: Permukaan yang miring memantulkan suara menjauh, jadi dinding yang menyudut sering terbaca lebih jauh dari kenyataannya atau tidak terbaca sama sekali. Kain dan busa menyerap suara. Benda yang terlalu dekat, di bawah beberapa sentimeter, juga tidak terbaca dengan benar.

Color Sensor

Berfungsi untuk membaca warna dan tingkat pantulan permukaan di bawahnya.



Sama seperti sebelumnya, sensor ini juga punya lampu-lampu sebagaimana terannotasi di gambar kanan, dengan `sensor.lights.on([100, 0, 0])` bisa kalian mati-nyalakan satu-satu.

Color Sensor

```
1 from pybricks.pupdevices import ColorSensor
2 from pybricks.parameters import Port, Color
3 from pybricks.tools import wait
4
5 sensor = ColorSensor(Port.D)
6
7 while True:
8     print(sensor.color(), sensor.reflection())
9     wait(200)
```

| Color Sensor

Color Methods

Perintah	Hasil
<code>sensor.color()</code>	Salah satu nilai <code>Color</code> , misal <code>Color.RED</code>
<code>sensor.reflection()</code>	Persentase pantulan, 0 sampai 100
<code>sensor.ambient()</code>	Persentase cahaya sekitar, 0 sampai 100
<code>sensor.hsv()</code>	Nilai hue, saturation, dan value mentah

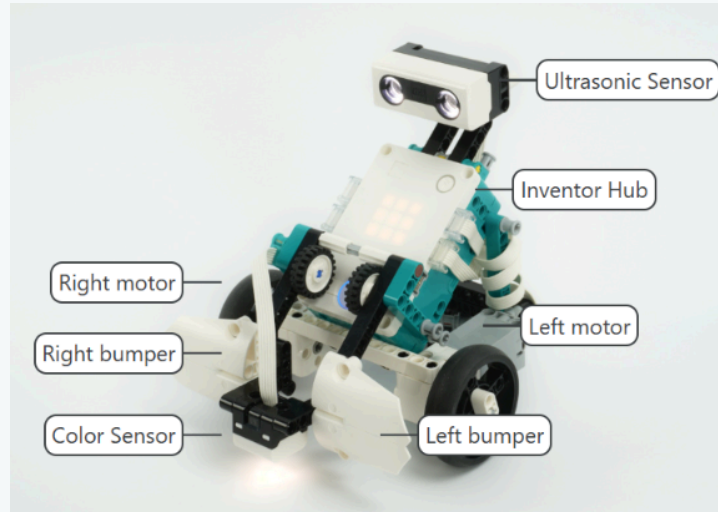
Color Notes

Untuk mengikuti garis, `reflection()` lebih berguna daripada `color()` karena hasilnya berupa angka yang bisa diukur, bukan kategori.

Jarak sensor ke lantai sangat berpengaruh. Beda dua milimeter sudah mengubah angka pantulan. Pasang sensor sedekat mungkin ke lantai dan jangan diubah-ubah setelah kalian mengukur nilai acuan.

Hub Buttons

Rancangan StarterBot resmi memakai tombol kiri dan kanan hub sebagai bumper mekanis. Batang LEGO menempel ke tombol, dan saat robot menabrak sesuatu, batang itu menekan tombol.



Hub Buttons

Ini berguna karena Ultrasonic Sensor punya sudut pandang relatif sempit. Benda tipis seperti kaki meja sering lolos dari deteksi, dan bumper bisa digunakan untuk menangkapnya.

```
1 from pybricks.hubs import InventorHub
2 from pybricks.parameters import Button
3 from pybricks.tools import wait
4
5 hub = InventorHub()
6
7 while True:
8     pressed = hub.buttons.pressed()
9     if Button.LEFT in pressed:
10         print("tabrakan di kiri")
11     if Button.RIGHT in pressed:
12         print("tabrakan di kanan")
```

Timings

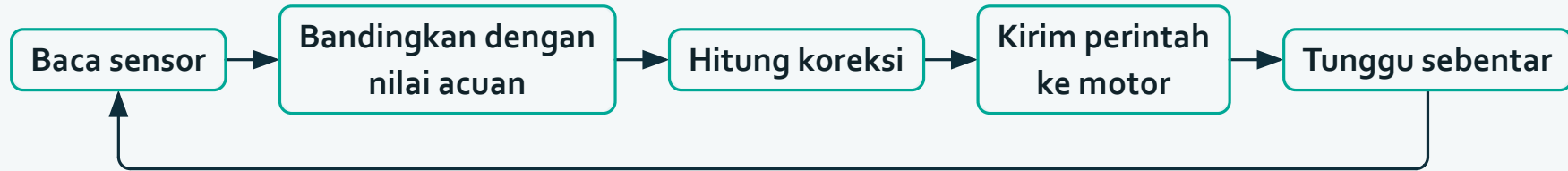
```
1 from pybricks.tools import wait, Stopwatch
2
3 wait(1000)                                # berhenti 1000 ms
4
5 timer = Stopwatch()
6 robot.straight(500)
7 print("butuh", timer.time(), "ms")
```

StopWatch mulai menghitung begitu dibuat. Metodenya `time()`, `pause()`, `resume()`, dan `reset()`.

I Loops

Sampai sini kalian menyuruh robot melakukan gerakan yang sudah ditentukan. Sekarang robot yang menentukan gerakannya sendiri berdasarkan pembacaan sensor.

Bentuknya selalu sama, apa pun sensornya:



Robot tidak pernah tahu posisinya yang sebenarnya. Yang dia tahu cuma selisih antara pembacaan sensor sekarang dan nilai yang dia inginkan, dan dia bergerak untuk mengecilkan selisih itu.

Loops

Stop Before Crash

Berikut contoh kode yang berhenti sebelum menabrak permukaan:

```
1 from pybricks.pupdevices import Motor, UltrasonicSensor
2 from pybricks.parameters import Port, Direction
3 from pybricks.robotics import DriveBase
4 from pybricks.tools import wait
5
6 left = Motor(Port.A, Direction.COUNTERCLOCKWISE)
7 right = Motor(Port.B, Direction.CLOCKWISE)
8 eyes = UltrasonicSensor(Port.C)
9
10 robot = DriveBase(left, right, wheel_diameter=56, axle_track=114)
11
12 robot.drive(150, 0)
```

Loops

```
13
14 while eyes.distance() > 200:
15     wait(10)
16
17 robot.stop()
18 print("berhenti pada jarak", eyes.distance(), "mm")
```

Ini harus kalian jalankan beberapa kali dan catat angka terakhirnya, karena angkanya tidak akan sama persis, dan tidak akan tepat 200 mm. Robot butuh waktu untuk membaca sensor, dan setelah `stop()` dipanggil robot masih meluncur sedikit.

Loops

Mengikuti garis

Contoh *proportional control*, cara paling sederhana membuat robot mengoreksi diri sendiri.

```
1 from pybricks.pupdevices import Motor, ColorSensor
2 from pybricks.parameters import Port, Direction
3 from pybricks.robotics import DriveBase
4 from pybricks.tools import wait
5
6 left = Motor(Port.A, Direction.COUNTERCLOCKWISE)
7 right = Motor(Port.B, Direction.CLOCKWISE)
8 sensor = ColorSensor(Port.D)
9
10 robot = DriveBase(left, right, wheel_diameter=56, axle_track=114)
11
12 BLACK = 9
```

Loops

```
13 WHITE = 85
14 threshold = (BLACK + WHITE) / 2
15
16 DRIVE_SPEED = 100
17 GAIN = 1.2
18
19 while True:
20     deviation = sensor.reflection() - threshold
21     turn_rate = GAIN * deviation
22     robot.drive(DRIVE_SPEED, turn_rate)
23     wait(10)
```

Robot tidak mengikuti tengah garis. Robot mengikuti tepi garis, tempat pantulan bernilai di antara hitam dan putih.

Loops

Posisi Sensor	Pantulan	deviation	Yang Dilakukan Robot
Terlalu ke putih	mendekati 85	positif	Membelok ke arah garis
Pas di tepi garis	sekitar 47	mendekati nol	Jalan lurus
Terlalu ke hitam	mendekati 9	negatif	Membelok menjauh dari garis

Nilai **BLACK** dan **WHITE** harus kalian ukur sendiri dengan program di 9.2, di lintasan yang akan dipakai, dengan pencahayaan ruangan saat itu. Angka 9 dan 85 di atas cuma titik awal.

GAIN menentukan seberapa keras robot mengoreksi. Terlalu kecil, robot memotong tikungan dan lepas dari garis. Terlalu besar, robot berkelok-kelok tajam dan bisa terlempar keluar. Naikkan sedikit demi sedikit sampai robot mengikuti garis dengan goyangan yang masih wajar, lalu berhenti di situ.

Loops

Ini bagian yang tidak bisa dihitung di atas kertas. Nilai yang benar bergantung pada berat robot, jenis ban, kecepatan, dan permukaan lintasan.

Penugasan



Buat sebuah robot maze solver. Tiap tim akan mendemokan hasil masing-masing **minggu depan**.

Tidak terdapat restriksi pada metode maupun desain dari robot, tetapi dipastikan bahwa masing-masing anggota kelompok memahami dan dapat menjelaskan mekanisme yang diterapkan pada robot.

I Additional Materials

Dokumentasi API lengkap ada di <https://docs.pybricks.com>. Halaman yang paling sering kalian buka:

Topik	Halaman Resmi
Motor	Motor
DriveBase	robotics
Sensor	pupdevices
Layar, tombol, IMU, speaker, baterai	Prime Hub / Inventor Hub
Port, Direction, Stop, Color, Button, Icon	parameters
wait dan StopWatch	tools
Satuan yang dipakai setiap perintah	Signals and Units

dan materi ini dapat diakses di:

<https://its.id/m/materi-robotika-rka-n>

(Ada referensi materi yang lebih lengkap juga di repo).